

Manuale Tecnico Book Recommender

Giuseppe Antonuccio, Valerio Gazziero, Alessandro Salvini

Gennaio 2026

Contents

1	Software Design	4
1.1	Panoramica generale del sistema	4
1.2	Struttura del progetto (multi-modulo Maven)	6
1.2.1	1.2.1 Modulo common (contratto di comunicazione)	6
1.2.2	Modulo serverBR	7
1.2.3	1.2.3 Modulo clientBR	7
1.3	Architettura software	8
1.3.1	Stile architetturale: Client-Server a livelli	8
1.3.2	Pattern e scelte progettuali	8
1.4	Protocollo di comunicazione Client Server	9
1.4.1	Trasporto e framing	9
1.4.2	Formato dei messaggi	10
1.4.3	Serializzazione/deserializzazione	10
1.5	Connessione al Database (PostgreSQL) e bootstrap dati	10
1.5.1	Configurazione	10
1.5.2	DBManager: responsabilità e sequenza di init	11
1.6	Traduzione entità relazionali in entità Java	11
1.6.1	Tabella ‘Libri’ in classe Java ‘Libro’	11
1.6.2	Tabella ‘UtentiRegistrati’ come rappresentazione Java	12
1.6.3	Tabella Librerie e Librerie Libri: rappresentazione in Java	13
1.6.4	Tabella ‘ValutazioniLibri’: mapping a operazioni e aggregati	13
1.6.5	Tabella ‘ConsigliLibri’: mapping a liste e conteggi	14
2	Algoritmi impiegati	14
2.1	Ricerca dei libri (query testuali case-insensitive)	14
2.1.1	Ricerca per titolo	14
2.1.2	Ricerca per autore	15
2.1.3	Ricerca per autore e anno	15
2.2	Ricerca avanzata (selezione del criterio in base agli input)	15
2.3	Bootstrap del database: creazione schema e seed dataset	16
2.3.1	Connessione e creazione automatica del database (PostgreSQL)	16

2.3.2	Applicazione dello schema (schema.sql)	16
2.3.3	Seed iniziale dei libri (import CSV robusto con righe multilinea)	17
2.4	Gestione delle librerie: algoritmo di inserimento Many-to-Many con deduplicazione	17
2.4.1	Aggiunta libri a libreria	18
2.4.2	Verifica “il libro è nelle mie librerie”	18
2.5	Valutazioni e statistiche: upsert e aggregazione	18
2.5.1	Inserimento/aggiornamento valutazione (Upsert)	18
2.5.2	Mappatura del voto globale in 5 criteri (modalità GUI)	18
2.5.3	Calcolo aggregati (distribuzione + media)	19
2.5.4	Costruzione output recensioni con suggerimenti	19
2.6	Suggerimenti di lettura: vincolo massimo e ranking per frequenza	19
2.6.1	Vincolo “max 3 suggerimenti” (applicativo)	20
2.6.2	Conteggio suggerimenti per visualizzazione libro (ranking)	20
3	Gestione della concorrenza	20
3.1	Modello di concorrenza del server: thread-per-connessione con thread pool	20
3.1.1	Creazione del pool	20
3.1.2	Accettazione e delega	21
3.2	Concorrenza per singola connessione: elaborazione sequenziale delle richieste	21
3.3	Isolamento e stato condiviso: cosa è thread-safe e cosa richiede attenzione	21
3.3.1	Componenti sostanzialmente stateless	21
3.3.2	Risorsa condivisa principale: connessione JDBC unica	22
3.4	Coerenza dei dati in concorrenza: vincoli DB e strategie applicative	22
3.5	Concorrenza lato client: singleton della connessione e serializzazione richiesta/risposta	23
3.6	Gestione risorse in ambiente concorrente	23
4	Logging	23
4.1	Obiettivi del logging	23
4.2	Modalità attuale: console logging (stdout/stderr)	24
4.3	Punti di log presenti nel server	24
4.3.1	Avvio del server e accettazione connessioni	24
4.3.2	Gestione eccezioni durante l’elaborazione richieste	24
4.3.3	Disconnessioni client (IOException silenziosa)	25
4.4	Punti di log presenti nel bootstrap del DB (DBManager)	25
4.5	Redirezione dei log su file (operativo)	25
4.6	Evoluzione prevista: logging strutturato con SLF4J (dipendenza presente)	26

5	Strumenti, Librerie e Linguaggi utilizzati	26
5.1	Linguaggi utilizzati	26
5.1.1	Java (JDK 17)	26
5.1.2	SQL (PostgreSQL dialect)	27
5.1.3	FXML (JavaFX Markup)	27
5.1.4	JSON (formato di scambio)	27
5.1.5	Properties e CSV	27
5.2	Strumenti di build e gestione progetto	27
5.2.1	Apache Maven (multi-modulo)	27
5.2.2	Maven Assembly Plugin (fat jar)	28
5.2.3	JavaFX Maven Plugin	28
5.2.4	Maven Dependency Plugin (copia runtime JavaFX)	28
5.3	Librerie di terze parti (con versioni)	28
5.3.1	Gson 2.11.0 (serializzazione JSON)	28
5.3.2	PostgreSQL JDBC Driver 42.7.3	28
5.3.3	SLF4J Simple 2.0.13	28
5.3.4	JavaFX 21.0.5 (client)	29
5.4	Configurazione e risorse di progetto	29
5.4.1	File di configurazione	29
5.4.2	Schema e dataset	29
5.5	Tecnologie standard Java (senza dipendenze esterne)	29
6	Limiti dell'applicazione	30
6.1	Limiti del protocollo di comunicazione (TCP + JSON line-based)	30
6.1.1	Assenza di versionamento del protocollo	30
6.1.2	Limitazione “una richiesta = una riga”	30
6.1.3	Assenza di identificatore di correlazione (correlation id)	30
6.2	Limiti di sicurezza	31
6.2.1	Trasporto non cifrato (assenza di TLS)	31
6.2.2	Gestione credenziali: password trattata come valore “hash” ma non hashata	31
6.2.3	Assenza di gestione sessione e autorizzazione robusta	31
6.2.4	Validazioni input parziali	32
6.3	Limiti di concorrenza e scalabilità	32
6.3.1	Thread pool non limitato (<code>newCachedThreadPool</code>)	32
6.3.2	Connessione JDBC unica condivisa tra thread	32
6.3.3	Assenza di transazioni per operazioni multi-step	32
6.3.4	Query senza paginazione (liste potenzialmente grandi)	33
6.4	Limiti del modello dati	33
6.4.1	Mancanza di metadati temporali	33
6.4.2	Campo note limitato a 256 caratteri	33
6.4.3	Vincolo “max 3 suggerimenti” non è garantito a livello DB	33
6.4.4	Nessuna prevenzione di suggerimenti duplicati nel payload	34
6.5	Limiti funzionali (feature non implementate o semplificate)	34
6.5.1	Gestione librerie parziale	34
6.5.2	Recensioni: una per utente e libro, senza storico	34

6.5.3	Assenza di funzionalità di recupero account	34
6.6	Limiti di usabilità, performance percepita e portabilità del client	35
6.6.1	Chiamate di rete sincrone su thread della UI	35
6.6.2	Connessione socket spesso ricreata	35
6.6.3	Portabilità JavaFX limitata da configurazione piattaforma	35
6.7	Limiti operativi di deployment	35
6.7.1	Dipendenza da PostgreSQL e privilegi di creazione database	35
6.7.2	Seed CSV non garantisce completezza e segnala poco gli errori	36
6.8	Sintesi dei limiti principali	36

1 Software Design

Questo capitolo descrive in modo approfondito come è progettato il progetto BookRecommender, coprendo:

- Struttura dell'applicazione(moduli, package, responsabilità)
- Architettura(client-server, livelli, flussi)
- Connessione al database(configurazione, inizializzazione, schema e seed)
- Traduzione delle entità relazionali in entità Java

1.1 Panoramica generale del sistema

Il progetto è costruito come *applicazione client-server* e si compone da:

- Client (JavaFX): gestisce interfaccia utente e interazione con l'utente (login/registrazione, ricerca libri, gestione librerie, recensioni)
- Server (Java SE): espone funzionalità applicative tramite socket TCP; contiene la logica di orchestrazione (dispatch delle richieste) e l'accesso ai dati (DAO)
- Database (PostgreSQL): persistenza dati (utenti, libri, librerie, valutazioni, suggerimenti)

La comunicazione avviene con un protocollo semplice su TCP: una richiesta JSON per riga per una risposta JSON per riga.

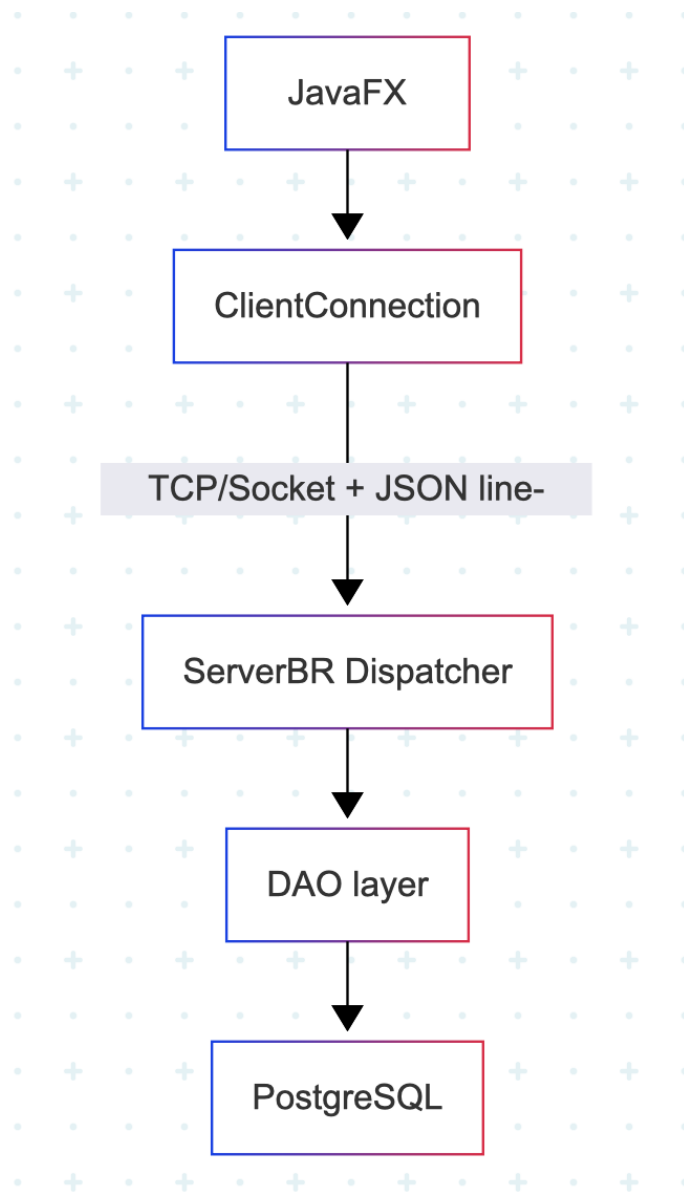


Figure 1: Schema concettuale ad alto livello

1.2 Struttura del progetto (multi-modulo Maven)

Il progetto è un multi-modulo Maven con root aggregator:

- BookRecommender (root pom): packaging ‘pom’, definisce moduli e versioni comuni (Java 17, JavaFX)
- common: classi condivise tra client e server
- serverBR: server concorrente con DAO JDBC e bootstrap DB
- clientBR: applicazione JavaFX che si connette al server

1.2.1 1.2.1 Modulo common (contratto di comunicazione)

Package: com.bookrecommender.common

Contiene gli oggetti “contrattuali” usati da entrambi i lati:

- Request
 - type: enum ‘RequestType’
 - payload: ‘Map<String, Object>’ con i parametri dell’operazione
- Response
 - ok : boolean (success/fail)
 - message: stringa d’errore in caso di fail
 - data: Map<String, Object> con risultato in caso di successo
- RequestType (enum)
 - include comandi “GUI oriented” (LOGIN, SEARCH BOOKS, ...)
 - include comandi “spec oriented” (CERCA TITOLO, VISUALIZZA LIBRO, ...)
 - obiettivo: compatibilità con più set di chiamate senza duplicare server.

Questa scelta rende il protocollo fortemente tipizzato sul campo operazione (‘RequestType’) e flessibile sui parametri (‘payload’ generico).

1.2.2 Modulo serverBR

Package principali:

- com.bookrecommender.server
 - ServerBR: main server, gestione socket e dispatch
- com.bookrecommender.server.dao
 - DBManager: connessione DB, schema, seed iniziale
 - BookDAO, UserDAO, LibraryDAO, RatingDAO, SuggestionDAO: accesso dati tramite JDBC
- com.bookrecommender.server.model
 - Libro: modello dati server-side (allineato al JSON)
- com.bookrecommender.server.tools
 - utility di supporto (creazione DB, import CSV, ecc.)

Risorse serverBR/src/main/resources:

- application.properties: parametri DB
- db/schema.sql: definizione tabelle e vincoli
- BooksDatasetClean.csv: dataset iniziale libri (seed)

1.2.3 1.2.3 Modulo clientBR

Package principali:

- com.bookrecommender.ui
 - MainApp: entrypoint JavaFX
 - Controller FXML: MainController, BookDetailsController, ecc.
- com.bookrecommender.client
 - ClientConnection: gestione socket e invio richieste
- com.bookrecommender.model
 - Libro: modello client-side (POJO con getter/setter)

Risorse clientBR/src/main/resources:

- ui/*.fxml: layout JavaFX
- app.properties: host/port server + (legacy) parametri db

1.3 Architettura software

1.3.1 Stile architetturale: Client-Server a livelli

L'architettura è suddivisa nei seguenti livelli: 1. Presentation Layer (Client JavaFX)

- controller UI
- gestione eventi (click, doppio click, dialog)
- rendering dei risultati (tabelle, dettagli libro, ecc.)

2. Application/Service Layer (ServerBR)

- punto unico di ingresso: 'dispatch(Request)'
- validazioni applicative
- orchestrazione delle chiamate ai DAO
- costruzione di Response coerenti col protocollo

3. Data Access Layer (DAO e DBManager)

- query SQL tramite JDBC
- mapping ResultSet → oggetti Java o strutture 'Map'
- gestione integrità (vincoli DB + logiche applicative)

4. Data Layer (PostgreSQL)

- persistenza e vincoli (PK, FK, UNIQUE, CHECK, CASCADE)

1.3.2 Pattern e scelte progettuali

(A) DAO pattern (Data Access Object): Ogni “area” di persistenza ha un DAO dedicato:

- BookDAO: ricerca e recupero libri
- UserDAO: registrazione/login
- LibraryDAO: librerie personali e join libri
- RatingDAO: inserimento valutazioni + aggregazioni
- SuggestionDAO: gestione suggerimenti e conteggi

Vantaggi:

- SQL centralizzato e separato dal server dispatcher

- mapping e query riusabili
- riduzione della complessità nel livello application

(B) Dispatcher centrale nel server: `ServerBR.dispatch(req)` implementa un routing a `switch(RequestType)`:

- unica porta d'ingresso della logica applicativa
- facile capire “quali operazioni esistono”
- facile mantenere compatibilità tra comandi GUI e comandi specifica

(C) Concorrenza server: thread pool: Il server accetta connessioni TCP e le gestisce con:

- `ExecutorService pool = Executors.newCachedThreadPool();`
- ogni socket client viene gestito da `handleClient(socket)` in un thread del pool

Implicazioni:

- più client possono lavorare in parallelo
- l'accesso al DB avviene tramite un'unica `Connection` statica (in `DBManager`)

1.4 Protocollo di comunicazione Client Server

1.4.1 Trasporto e framing

Trasporto: TCP socket Framing: line-based

- il client invia una riga JSON terminata da carattere newline
- il server risponde con una riga JSON terminata da carattere newline

Questa scelta evita la complessità di:

- lunghezze prefissate
- multipart payload
- protocolli binari

Di contro, ciò richiede l'imposizione di una regola: ogni messaggio deve stare su una singola riga (il JSON deve essere serializzato senza newline).

1.4.2 Formato dei messaggi

Request

- type: enum RequestType (obbligatorio)
- payload: mappa chiave/valore (opzionale)

Response

- ok: true/false
- message: testo errore se ok=false
- data: risultati se ok=true

1.4.3 Serializzazione/deserializzazione

Entrambi i lati usano Gson:

- server: new Gson() in ServerBR
- client: new Gson() in ClientConnection e nei controller (per parsing liste)

Implicazioni:

- I POJO (es. Libro) devono avere campi coerenti coi nomi JSON
- Nel client, spesso si fa conversione tramite TypeToken quando serve ricostruire List<Libro> da una Map

1.5 Connessione al Database (PostgreSQL) e bootstrap dati

1.5.1 Configurazione

Nel server, i parametri DB risiedono in serverBR/src/main/resources/application.properties:

- db.url=jdbc:postgresql://localhost:5432/bookrecommender
- db.user=postgres
- db.password=postgres

La configurazione è sovrascrivibile tramite variabili d'ambiente:

- BR DB URL
- BR DB USER
- BR DB PASSWORD

1.5.2 DBManager: responsabilità e sequenza di init

DBManager.init() è l'entrypoint di bootstrap DB chiamata dal server all'avvio (ServerBR.start()).

Esegue, in ordine:

1. Caricamento proprietà: legge application.properties dal classpath e applica override da variabili d'ambiente
2. Connessione JDBC (DriverManager.getConnection(url,user,pwd))
3. Creazione automatica del database se mancante (solo Postgres): se la connessione fallisce l'URL è jdbc:postgresql://... tenta di connettersi a .../postgres, controlla pg database per vedere se il DB target esiste e, se non esiste, esegue CREATE DATABASE "bookrecommender"
4. Applicazione schema: legge db/schema.sql, esegue ogni statement con Statement.execute(...) ed usa CREATE TABLE IF NOT EXISTS
5. Seed iniziale dei libri: esegue SELECT COUNT(*) FROM Libri. Se Libri è vuota, legge BooksDatasetClean.csv ed inserisce in Libri(titolo,autore,anno,genere,descrizione)

1.6 Traduzione entità relazionali in entità Java

Il database definisce queste tabelle:

- UtentiRegistrati
- Libri
- Librerie
- Librerie Libri(tabella ponte many-to-many)
- ValutazioniLibri(chiave composta)
- ConsigliLibri(chiave composta tripla)

1.6.1 Tabella 'Libri' in classe Java 'Libro'

Versione relazionale:

- id SERIAL PRIMARY KEY
- titolo TEXT NOT NULL
- autore TEXT NOT NULL
- anno INT (nullable)
- genere TEXT (nullable)
- descrizione TEXT (nullable)

Java (server)

- `com.bookrecommender.server.model.Libro`
 - `public int id;`
 - `public String titolo;`
 - `public String autore;`
 - `public Integer anno;`
 - `public String genere;`
 - `public String descrizione;`

Java (client)

- `com.bookrecommender.model.Libro` con `getter/setter` e `Integer anno`

Mapping JDBC (`BookDAO.map`)

- `id: rs.getInt("id")`
- `titolo: rs.getString("titolo")`
- `autore: rs.getString("autore")`
- `anno: rs.getObject("anno")`
- `genere: rs.getString("genere")`
- `descrizione: rs.getString("descrizione")`

1.6.2 Tabella ‘UtentiRegistrati’ come rappresentazione Java

Versione Relazionale

- `id SERIAL PK`
- `nome, cognome NOT NULL`
- `codice fiscale UNIQUE`
- `email UNIQUE`
- `userid UNIQUE`
- `password hash NOT NULL`

Nel codice non esiste una classe ‘Utente’: la scelta progettuale è gestire l’utente come record procedurale tramite `UserDAO` con ritorno del solo `id` (e del nome in login)

Funzioni principali:

- `register(...)`: `‘INSERT ... RETURNING id’`
- `login(useridOrEmail, passwordHash)`: `SELECT id ...`
- `getNomeById(id)`: `SELECT nome ...`

1.6.3 Tabella Librerie e Librerie Libri: rappresentazione in Java

Relazioni:

- UtentiRegistrati (1) : (N) Librerie
 - FK: Librerie.utente id REFERENCES UtentiRegistrati(id) ON DELETE CASCADE
- Librerie (N) : (N) Libri tramite Librerie Libri
 - PK composta: (libreria id, libro id) entrambe FK con clausola ON DELETE CASCADE

Rappresentazione Java

- Non esiste la classe Libreria
- LibraryDAO‘ espone metodi che lavorano con:
 - int utenteId
 - int libreriaId
 - String nome
 - List<Integer> libroIds

1.6.4 Tabella ‘ValutazioniLibri’: mapping a operazioni e aggregati

Versione Relazionale:

- PK composta: (utente id, libro id)
- 5 campi voto: stile, contenuto, gradevolezza, originalita, edizione (1..5)
- voto finalecalcolato in DB come media
- note

Java:

- Non c’è una entity Valutazione
- RatingDAO.inserisci(...) fa update ed insert logico:
- INSERT ... ON CONFLICT(utente id, libro id) DO UPDATE SET ...

1.6.5 Tabella ‘ConsigliLibri’: mapping a liste e conteggi

Versione Relazionale

- PK tripla(utente id, libro id, suggerito libro id)‘
- Rappresenta: “per il libro X, l’utente U suggerisce il libro Y”

Versione Java:

- Nessuna entity ‘Consiglio‘
- SuggestionDAO lavora con:
 - List<Integer> suggeriti (max 3) per l’inserimento
 - List<String> titoli per mostrare i consigli nelle review
 - List<Map<String, Object>> per conteggi aggregati (suggerito libro id, n)

2 Algoritmi impiegati

In questa sezione vengono descritti in modo dettagliato gli algoritmi e le procedure principali implementate nel sistema, con riferimento alle componenti server (**serverBR**) e, dove rilevante, alle scelte di protocollo condivise (**common**).

2.1 Ricerca dei libri (query testuali case-insensitive)

La funzionalità di ricerca è implementata nel **BookDAO** e si basa su query SQL parametrizzate (*PreparedStatement*) per evitare injection e per separare la logica applicativa dai dettagli di esecuzione.

2.1.1 Ricerca per titolo

Il metodo `searchByTitle(q)` esegue una ricerca per sottostringa sul campo `titolo`.

- **Normalizzazione case-insensitive:** la query applica `LOWER()` sia alla colonna sia al parametro, ottenendo una comparazione indipendente da maiuscole/minuscole.
- **Matching per sottostringa:** il parametro viene costruito come `%q%`, quindi l’algoritmo semantico della ricerca è “contiene”.
- **Ordinamento:** i risultati vengono ordinati per `titolo` per garantire una presentazione stabile lato client.

In termini pratici, la query corrisponde a:

```
SELECT *
FROM Libri
WHERE LOWER(titolo) LIKE LOWER('%q%')
ORDER BY titolo;
```

Osservazioni prestazionali La condizione `LIKE 'q'` (con wildcard iniziale) tipicamente impedisce l'uso efficiente di un indice B-Tree standard sulla colonna, portando a scansioni più estese all'aumentare del dataset. Nel contesto del progetto, la scelta privilegia semplicità e robustezza funzionale.

2.1.2 Ricerca per autore

La ricerca per autore `searchByAuthor(q)` replica lo stesso schema logico della ricerca per titolo, applicata al campo `autore` e mantenendo l'ordinamento per titolo.

2.1.3 Ricerca per autore e anno

La ricerca combinata `searchByAuthorYear-autore, anno)` applica un filtro composto:

- **autore** come sottostringa case-insensitive
- **anno** come uguaglianza esatta (`anno = ?`)

La logica implementa quindi una *congiunzione* (AND) tra un filtro testuale e un filtro numerico.

2.2 Ricerca avanzata (selezione del criterio in base agli input)

La richiesta `SEARCH_BOOKS_ADVANCED` (server: `ServerBR.dispatch`) implementa una **strategia di selezione del criterio** basata su priorità e presenza dei campi compilati:

1. Se `titolo` è non vuoto, viene eseguita la ricerca per titolo.
2. Altrimenti, se `autore` è non vuoto e `anno` è non vuoto:
 - tenta il parsing intero di `anno`
 - se il parsing riesce, usa la ricerca autore+anno
 - se il parsing fallisce, degrada a ricerca per autore
3. Altrimenti, se solo `autore` è non vuoto, ricerca per autore.
4. Altrimenti, nessun criterio: ritorna tutti i libri (implementato come ricerca per titolo con stringa vuota).

Questa logica garantisce:

- comportamento deterministico (stesse condizioni → stessa query)
- robustezza rispetto a input non numerici nel campo anno
- un fallback sensato quando l'utente non inserisce criteri

2.3 Bootstrap del database: creazione schema e seed dataset

La classe `DBManager` implementa una procedura di inizializzazione del database composta da tre fasi principali, eseguite all'avvio del server tramite `DBManager.init()`.

2.3.1 Connessione e creazione automatica del database (PostgreSQL)

L'algoritmo di connessione segue questo flusso:

1. Legge `application.properties` (e applica override tramite variabili d'ambiente).
2. Tenta `DriverManager.getConnection(url, user, pwd)`.
3. Se fallisce e l'URL è PostgreSQL:
 - ricava host/porta e nome database dalla URL
 - si connette al database amministrativo `postgres`
 - verifica l'esistenza del database target interrogando `pg_database`
 - se mancante, esegue `CREATE DATABASE "dbname"`
 - ritenta la connessione al database target

Questa procedura consente un setup “zero-touch” in ambiente locale: se il DB non esiste, viene creato automaticamente senza intervento manuale.

2.3.2 Applicazione dello schema (schema.sql)

Lo schema è caricato da risorsa `db/schema.sql` e applicato tramite:

- lettura del file completo in memoria
- split in statement tramite separatore ;
- esecuzione sequenziale con `Statement.execute(sql)`

La scelta di esecuzione sequenziale è deliberata: garantisce che vincoli e tabelle vengano creati nell'ordine definito nel file.

2.3.3 Seed iniziale dei libri (import CSV robusto con righe multilinea)

Il popolamento iniziale avviene **solo se** la tabella `Libri` è vuota:

```
SELECT COUNT(*) FROM Libri;
```

Se il conteggio è pari a zero, viene letto `BooksDatasetClean.csv`. L'import è progettato per gestire CSV con:

- delimitatore `;` (punto e virgola)
- campi tra doppi apici
- descrizioni potenzialmente **multilinea** (newline all'interno del campo tra apici)

Per ottenere robustezza, la procedura:

1. legge l'intestazione e individua dinamicamente le colonne (es. `Title`, `Author`, `Year`, ecc.) usando `match case-insensitive` per contenimento.
2. usa un buffer incrementale:
 - accumula righe fino a quando il numero di `;` nel buffer suggerisce che il record sia completo (almeno `nColonne-1`)
 - solo allora effettua il parsing del record
3. effettua il parsing con un automa semplice "inQuotes":
 - alterna stato quando incontra `"`
 - tratta `;` come separatore solo quando `inQuotes=false`
 - ignora newline fuori dai quotes, preserva contenuto dentro i quotes
4. normalizza e valida i dati minimi:
 - scarta record con titolo vuoto
 - per l'anno: converte a intero se possibile, altrimenti imposta `NULL`
5. inserisce i record in modalità batch con `PreparedStatement.addBatch()` e `executeBatch()`.

Perché il batch L'inserimento batch riduce i round-trip verso il DB e rende l'import significativamente più efficiente rispetto a un ciclo di insert singole.

2.4 Gestione delle librerie: algoritmo di inserimento Many-to-Many con deduplicazione

La relazione tra librerie e libri è modellata tramite tabella ponte `Librerie_Libri` (PK composta).

2.4.1 Aggiunta libri a libreria

L'algoritmo di `LibraryDAO.aggiungiLibri(libreriaId, libroIds)` è:

- per ogni `libroId` crea una riga di relazione (`libreria_id`, `libro_id`)
- usa batch insert
- applica `ON CONFLICT DO NOTHING` per evitare duplicati

Questa scelta realizza una deduplicazione **idempotente**: ripetere la stessa richiesta non altera lo stato oltre la prima applicazione.

2.4.2 Verifica “il libro è nelle mie librerie”

La funzione `libroNelleMieLibrerie(utenteId, libroId)` implementa un controllo di appartenenza con join:

- unisce `Librerie` con `Librerie_Libri` sulla chiave della libreria
- filtra per `utente_id` e `libro_id`
- ritorna vero se esiste almeno una riga

Il controllo è fondamentale perché viene usato come **guardia applicativa** per le recensioni: un utente può recensire solo libri presenti nelle sue librerie.

2.5 Valutazioni e statistiche: upsert e aggregazione

Le valutazioni sono memorizzate in `ValutazioniLibri` con PK composta (`utente_id`, `libro_id`), quindi ogni utente può avere **al massimo una valutazione** per libro.

2.5.1 Inserimento/aggiornamento valutazione (Upsert)

`RatingDAO.inserisci(...)` implementa un upsert tramite `ON CONFLICT`:

- se la riga non esiste: `INSERT`
- se esiste già (stessa PK composta): `UPDATE` dei campi valutativi e note

Questo garantisce:

- unicità della recensione per coppia (utente, libro)
- possibilità di modifica nel tempo senza creare duplicati

2.5.2 Mappatura del voto globale in 5 criteri (modalità GUI)

Nel caso della richiesta `RATE_BOOK` (GUI), il sistema raccoglie un singolo voto globale (1–5) e lo mappa sui cinque criteri (**stile**, **contenuto**, **gradevolezza**, **originalità**, **edizione**) assegnando lo stesso valore a ciascuno. Questa scelta mantiene compatibilità con la struttura dati più ricca prevista dalla tabella e dalle funzioni “di specifica”.

2.5.3 Calcolo aggregati (distribuzione + media)

La funzione `RatingDAO.aggregati(libroId)` calcola, per ciascun campo (incluso `voto_finale`), una struttura composta da:

- conteggi per ciascun voto 1..5 (`c1..c5`) usando `COUNT(*) FILTER (WHERE ...)`
- media arrotondata a 2 decimali con `ROUND(AVG(...),2)`

L'algoritmo:

1. definisce l'insieme di campi target
2. per ogni campo, genera una query parametrica (template) sostituendo il nome del campo
3. esegue la query e costruisce una mappa `{c1..c5, media}`
4. inserisce la mappa sotto chiave corrispondente al campo (es. `"stile"`, `"voto_finale"`)

2.5.4 Costruzione output recensioni con suggerimenti

La richiesta `BOOK_STATS_AND_REVIEWS` unisce due sorgenti informative:

- aggregati per `voto_finale` (media e numero valutazioni)
- lista recensioni (`utente_id`, `voto_finale`, `note`)

Per ciascuna recensione, il server:

1. costruisce una stringa formattata con il voto e le note
2. recupera i titoli dei libri suggeriti da quello specifico utente per quel libro
3. se presenti, li concatena all'output della recensione

Il risultato è una lista di stringhe già pronte per la UI, evitando composizioni complesse lato client.

2.6 Suggerimenti di lettura: vincolo massimo e ranking per frequenza

I suggerimenti sono memorizzati in `ConsigliLibri` con PK tripla (`utente_id`, `libro_id`, `suggerito_libro_id`).

2.6.1 Vincolo “max 3 suggerimenti” (applicativo)

Il metodo `SuggestionDAO.inserisci(utenteId, libroId, suggeriti)` applica la seguente strategia:

1. elimina **tutti** i suggerimenti esistenti per la coppia (utente, libro)
2. verifica `suggeriti.size() <= 3`; se no, lancia eccezione
3. inserisce i nuovi suggerimenti in batch

Questa implementazione produce un comportamento “sostituisci lista” (replace) invece di “aggiungi” (append), rendendo immediata la sincronizzazione tra UI e stato persistente.

2.6.2 Conteggio suggerimenti per visualizzazione libro (ranking)

La funzione `conteggioPerLibro(libroId)` calcola quante volte ciascun libro è stato suggerito come raccomandazione per il libro target:

- raggruppa per `suggerito_libro_id`
- conta le occorrenze
- ordina per conteggio decrescente

In pratica realizza un **ranking per popolarità** basato sul numero di suggerimenti ricevuti.

3 Gestione della concorrenza

Il sistema gestisce concorrenza principalmente lato server, permettendo connessioni multiple simultanee. Lato client, l’uso tipico è seriale (invio richiesta, attesa risposta), coerente con la natura interattiva di una GUI.

3.1 Modello di concorrenza del server: thread-per-connessione con thread pool

Il server (`ServerBR`) implementa un loop di accettazione connessioni su `ServerSocket` e delega la gestione di ciascun client ad un pool di thread.

3.1.1 Creazione del pool

Il pool viene creato come:

```
ExecutorService pool = Executors.newCachedThreadPool();
```

Caratteristiche principali:

- **crescita dinamica**: crea nuovi thread quando necessario (se non ci sono thread liberi).

- **riuso**: thread inattivi vengono riutilizzati per nuove connessioni.
- **adatto a carichi variabili**: utile quando il numero di client può oscillare.

3.1.2 Accettazione e delega

Per ogni `accept()`:

- viene creato un task che invoca `handleClient(socket)`
- il task viene sottomesso al pool tramite `pool.submit(...)`

Quindi, a livello logico:

```
while(true):
    s = accept()
    pool.submit( handleClient(s) )
```

3.2 Concorrenza per singola connessione: elaborazione sequenziale delle richieste

All'interno di `handleClient` il server legge dal socket riga-per-riga:

- ogni riga corrisponde a una richiesta JSON completa
- per ciascuna richiesta viene calcolata una risposta
- la risposta viene scritta e flushata immediatamente

È importante notare che, **all'interno della stessa connessione**, le richieste sono gestite in modo **sequenziale** (non parallelizzate). Questo elimina problemi di interleaving delle risposte per lo stesso client e semplifica la gestione del protocollo.

Implicazione La concorrenza è quindi **tra connessioni** (client diversi), non **dentro la stessa connessione** (stesso client).

3.3 Isolamento e stato condiviso: cosa è thread-safe e cosa richiede attenzione

3.3.1 Componenti sostanzialmente stateless

I DAO (`BookDAO`, `UserDAO`, `LibraryDAO`, `RatingDAO`, `SuggestionDAO`) non mantengono stato mutabile interno per-request:

- creano *PreparedStatement* locali
- producono risultati locali (liste/mappe) restituiti al chiamante

Questo riduce la superficie di bug legati a race condition su campi condivisi.

3.3.2 Risorsa condivisa principale: connessione JDBC unica

`DBManager` conserva una **singola** `Connection` statica condivisa da tutti i thread tramite `DBManager.get()`.

Come viene mitigato nel progetto Nel codice:

- le query sono brevi e atomiche (auto-commit attivo)
- ogni *PreparedStatement* è creato ed usato dentro un blocco **try-with-resources**, quindi viene chiuso subito dopo l'uso

Nota tecnica importante A livello di specifica JDBC, una `Connection` condivisa tra thread non è garantita thread-safe in generale. Nel contesto del progetto (carichi moderati, query semplici, auto-commit), questa scelta può risultare funzionante, ma è un punto progettuale da evidenziare in un manuale tecnico.

Evoluzione consigliata (se si volesse rendere il server più robusto)

- usare una connessione per thread (es. tramite `ThreadLocal<Connection>`)
- oppure adottare un connection pool (es. HikariCP) con acquisizione/ri-lascio per operazione
- introdurre transazioni esplicite per operazioni multi-step che devono essere atomiche

3.4 Coerenza dei dati in concorrenza: vincoli DB e strategie applicative

Il progetto si affida a:

- **vincoli di chiave primaria e univocità** (per evitare duplicati)
- **ON CONFLICT** (upsert o deduplicazione idempotente)
- **controlli applicativi** (es. max 3 suggerimenti; recensione permessa solo se il libro è in libreria dell'utente)

In particolare:

- `ValutazioniLibri`: `ON CONFLICT(utente_id, libro_id) DO UPDATE` evita race su doppie insert e mantiene un solo record per coppia.
- `Librerie_Libri`: `ON CONFLICT DO NOTHING` rende l'operazione ripetibile senza duplicare righe.

3.5 Concorrenza lato client: singleton della connessione e serializzazione richiesta/risposta

Lato client, `ClientConnection` è realizzato come singleton (metodo `getInstance()` sincronizzato). L'uso previsto dalla GUI è:

- inviare una richiesta
- attendere la risposta
- aggiornare la UI

Poiché il metodo `send(...)` scrive sul socket e legge una singola riga di risposta, l'approccio è naturalmente **sincrono**. In caso di chiamate concorrenti (non tipiche in JavaFX se non si introducono thread di background), sarebbe opportuno serializzare l'accesso a `send` (ad esempio rendendolo **synchronized**) per evitare interleaving di richieste/risposte sullo stesso stream.

3.6 Gestione risorse in ambiente concorrente

Il server usa `try-with-resources` per stream del socket:

- chiusura automatica di `BufferedReader/BufferedWriter`
- chiusura del socket al termine della comunicazione o in caso di errore

Questo è cruciale in concorrenza perché evita leak di descrittori di file e mantiene stabile il numero di risorse di sistema anche con molti client che entrano/escono.

4 Logging

In questa sezione viene descritta la strategia di logging adottata dal progetto, cioè **dove** vengono generati messaggi di log, **con quale scopo**, **come** vengono prodotti (stdout/stderr) e quali sono le implicazioni operative in fase di debug e manutenzione.

4.1 Obiettivi del logging

Il logging nel progetto BookRecommender ha finalità principalmente **diagnostiche**:

- **Avvio e stato del servizio**: confermare che il server sia in ascolto e su quale porta.
- **Bootstrap del database**: evidenziare creazione del database, applicazione schema e seed iniziale.
- **Gestione errori runtime**: tracciare eccezioni in fase di elaborazione richieste o accesso dati.

- **Troubleshooting comunicazione:** rilevare problemi di connessione o protocollo (richieste non parsabili, disconnessioni).

4.2 Modalità attuale: console logging (stdout/stderr)

Nel codice corrente il logging è implementato tramite:

- `System.out.println(...)` per messaggi informativi.
- `e.printStackTrace()` per errori ed eccezioni inattese (stampa su `stderr`).

Questa scelta è semplice e adatta a un progetto didattico o a un ambiente di sviluppo locale, perché:

- non richiede configurazioni esterne;
- rende immediata l'osservabilità durante l'esecuzione da IDE/terminale;
- permette redirectione standard verso file (a livello OS) senza cambiare codice.

4.3 Punti di log presenti nel server

4.3.1 Avvio del server e accettazione connessioni

All'avvio, il server stampa un messaggio informativo su stdout:

ServerBR in ascolto sulla porta <port>

Questo messaggio è fondamentale per verificare rapidamente che:

- il server sia partito senza eccezioni;
- la porta di ascolto sia corretta (default 5555 o valore passato da argomento `args[0]`).

4.3.2 Gestione eccezioni durante l'elaborazione richieste

Durante il loop di lettura delle richieste (una riga JSON per volta), il server incapsula la deserializzazione e il dispatch in un blocco `try/catch`. In caso di errore:

- viene stampata la stacktrace completa tramite `e.printStackTrace()`;
- viene ritornata al client una `Response.error(...)` con un messaggio sintetico.

Questo meccanismo separa due esigenze:

- **osservabilità lato server:** stacktrace completa per debug;
- **robustezza lato client:** risposta strutturata in JSON anche in caso di failure, evitando blocchi o deadlock del protocollo.

Nota sul canale di output `printStackTrace()` scrive tipicamente su `stderr`, mentre i messaggi informativi scrivono su `stdout`. Ciò consente, se necessario, di separare flussi informativi e flussi di errore con redirectione OS.

4.3.3 Disconnessioni client (`IOException` silenziata)

Nel metodo `handleClient`, l'`IOException` esterna viene attualmente ignorata:

```
} catch (IOException ignored) {  
}
```

Questa scelta evita rumore di log per eventi fisiologici (client che chiudono la connessione). Tuttavia, in un contesto di diagnostica più spinta, può essere utile loggare almeno:

- la chiusura della connessione (normale vs anomala),
- l'indirizzo remoto (`socket.getRemoteSocketAddress()`),
- il motivo (messaggio eccezione).

4.4 Punti di log presenti nel bootstrap del DB (`DBManager`)

La classe `DBManager` produce messaggi espliciti, riconoscibili tramite prefissi tra parentesi quadre, ad esempio:

- **[DB]**: creazione automatica del database se mancante.
- **[SEED]**: import iniziale dei libri dal CSV.

Esempi:

```
[DB] Database 'bookrecommender' creato automaticamente.  
[SEED] BooksDatasetClean.csv non trovato, niente import iniziale.  
[SEED] Import iniziale completato. Libri inseriti: <n>
```

Valore pratico dei prefissi I prefissi rendono immediata la filtrazione a colpo d'occhio (anche senza livelli di log formali) e forniscono una convenzione uniforme per comprendere l'origine del messaggio.

4.5 Redirezione dei log su file (operativo)

Poiché il logging è su `stdout/stderr`, la persistenza su file è gestibile da terminale:

- su sistemi Unix-like (Linux/macOS):

```
java -jar serverBR-jar-with-dependencies.jar > server.out 2> server.err
```

- per un unico file con `stdout+stderr`:

```
java -jar serverBR-jar-with-dependencies.jar > server.log 2>&1
```

4.6 Evoluzione prevista: logging strutturato con SLF4J (dipendenza presente)

Nel modulo `serverBR` è dichiarata una dipendenza a `slf4j-simple` (SLF4J). Nel codice corrente i logger SLF4J non sono ancora utilizzati (si usano `System.out` e `printStackTrace`), ma la presenza della libreria rende naturale un'evoluzione verso:

- livelli (INFO, WARN, ERROR, DEBUG);
- messaggi consistenti e parametrizzati;
- configurazione runtime del livello di verbosità.

Configurazione tipica di `slf4j-simple` Qualora si introducano logger SLF4J nel codice, il livello di log può essere controllato via proprietà JVM, ad esempio:

```
-Dorg.slf4j.simpleLogger.defaultLogLevel=info  
-Dorg.slf4j.simpleLogger.showDateTime=true
```

Indicazione progettuale L'introduzione di SLF4J consentirebbe di trasformare i prefissi manuali [DB], [SEED] in logger o categorie (es. `com.bookrecommender.server.dao.DBManager`) mantenendo una migliore scalabilità del debugging.

5 Strumenti, Librerie e Linguaggi utilizzati

Questa sezione elenca in modo dettagliato i linguaggi, le librerie e gli strumenti di build/runtime impiegati dal progetto, includendo anche file di configurazione e risorse.

5.1 Linguaggi utilizzati

5.1.1 Java (JDK 17)

Il progetto è sviluppato in **Java 17**. La versione è impostata nel `pom.xml` root tramite:

```
<maven.compiler.source>17</maven.compiler.source>  
<maven.compiler.target>17</maven.compiler.target>
```

Java 17 viene utilizzato per:

- networking TCP (`java.net.Socket`, `java.net.ServerSocket`);
- I/O buffering (`BufferedReader`, `BufferedWriter`);
- concorrenza (`ExecutorService`, `Executors`);
- accesso DB (`java.sql.*`) via JDBC;
- strutture dati (`Map`, `List`) per payload e risultati.

5.1.2 SQL (PostgreSQL dialect)

Il livello di persistenza usa SQL conforme a PostgreSQL, in particolare:

- DDL e vincoli in `db/schema.sql`;
- DML via query nei DAO;
- costrutti specifici PostgreSQL come `ON CONFLICT` e interrogazione a `pg_database` (per auto-creazione DB).

5.1.3 FXML (JavaFX Markup)

L'interfaccia grafica del client è descritta in file `.fxml`:

- definizione layout e componenti UI;
- binding con controller Java tramite `fx:controller` e `@FXML`.

5.1.4 JSON (formato di scambio)

Client e server comunicano tramite messaggi JSON, con framing **riga-per-riga**:

- 1 richiesta = 1 riga JSON;
- 1 risposta = 1 riga JSON.

5.1.5 Properties e CSV

- `application.properties` per configurazione DB lato server.
- `app.properties` per host/porta lato client.
- `BooksDatasetClean.csv` come dataset di seed dei libri.

5.2 Strumenti di build e gestione progetto

5.2.1 Apache Maven (multi-modulo)

Il progetto è un **multi-modulo Maven** con root aggregator:

- `common`: classi condivise (`Request/Response/RequestType`).
- `serverBR`: server + DAO + bootstrap DB.
- `clientBR`: client JavaFX + connessione socket.

Vantaggi della struttura multi-modulo

- separazione netta tra componenti riusabili (`common`) e runtime distinti (`client/server`);
- versionamento interno coerente (stessa `version` per moduli);
- build indipendenti ma coordinate.

5.2.2 Maven Assembly Plugin (fat jar)

Sia client che server usano `maven-assembly-plugin` per produrre un JAR eseguibile con dipendenze incluse (`jar-with-dependencies`):

- semplifica l'esecuzione senza dover gestire manualmente classpath complessi;
- utile per consegna e demo.

5.2.3 JavaFX Maven Plugin

Nel modulo `clientBR` è presente `javafx-maven-plugin`, che consente l'avvio diretto con:

```
mvn javafx:run
```

5.2.4 Maven Dependency Plugin (copia runtime JavaFX)

Il client usa `maven-dependency-plugin` per copiare i jar JavaFX in `target/lib`, facilitando esecuzioni con `--module-path` quando necessario.

5.3 Librerie di terze parti (con versioni)

Le versioni qui riportate sono quelle dichiarate nei `pom.xml` dei moduli.

5.3.1 Gson 2.11.0 (serializzazione JSON)

Presente in `common`, `serverBR` e `clientBR`. Utilizzato per:

- serializzazione `Request` e `Response`;
- conversione di liste/oggetti (`Libro`) da/verso JSON;
- parsing robusto e veloce di payload a mappa.

5.3.2 PostgreSQL JDBC Driver 42.7.3

Presente nel modulo `serverBR`. Utilizzato per:

- connessione a PostgreSQL tramite `DriverManager`;
- esecuzione query e statement (`PreparedStatement`, `ResultSet`);
- supporto a funzioni PostgreSQL (es. `ON CONFLICT`).

5.3.3 SLF4J Simple 2.0.13

Presente nel modulo `serverBR`. Nel codice corrente non è ancora usato come API di logging (si usa console logging), ma costituisce la base per una migrazione verso logging con livelli e configurazione runtime.

5.3.4 JavaFX 21.0.5 (client)

Il client usa i moduli JavaFX:

- `javafx-controls`
- `javafx-fxml`
- `javafx-graphics`
- `javafx-base`

Nota sulla piattaforma Nel `pom.xml` del client è definita la proprietà:

```
<javafx.platform>win</javafx.platform>
```

Questa proprietà seleziona i jar nativi JavaFX (classifier) per Windows; per Linux/macOS va adeguata (`linux` o `mac`) in base all'ambiente di esecuzione.

5.4 Configurazione e risorse di progetto

5.4.1 File di configurazione

- `serverBR/src/main/resources/application.properties`: URL, user, password DB; supporta override con variabili d'ambiente `BR_DB_URL`, `BR_DB_USER`, `BR_DB_PASSWORD`.
- `clientBR/src/main/resources/app.properties`: host e porta del server (`server.host`, `server.port`).

5.4.2 Schema e dataset

- `serverBR/src/main/resources/db/schema.sql`: DDL e vincoli.
- `serverBR/src/main/resources/BooksDatasetClean.csv`: dataset per import iniziale nella tabella `Libri`.

5.5 Tecnologie standard Java (senza dipendenze esterne)

Oltre alle librerie di terze parti, il progetto fa uso estensivo delle API standard:

- **Networking**: `java.net`
- **Concorrenza**: `java.util.concurrent`
- **Persistenza**: `java.sql`
- **I/O**: `java.io`

Motivazione L'adozione di API standard riduce l'overhead di configurazione e facilita la portabilità, mantenendo la complessità entro i confini richiesti dal progetto.

6 Limiti dell'applicazione

Questa sezione descrive in modo sistematico i principali limiti dell'applicazione BookRecommender, distinguendo tra:

- **limiti architetturali e tecnologici** (scalabilità, concorrenza, protocollo, persistenza);
- **limiti di sicurezza** (autenticazione, trasporto, gestione credenziali);
- **limiti funzionali** (feature mancanti o semplificate);
- **limiti di usabilità e portabilità** (client JavaFX, performance percepita).

6.1 Limiti del protocollo di comunicazione (TCP + JSON line-based)

La comunicazione client-server avviene tramite socket TCP con messaggi JSON **una riga per volta**. Questa scelta semplifica implementazione e parsing, ma introduce limiti strutturali.

6.1.1 Assenza di versionamento del protocollo

- Non è presente un campo `protocolVersion` nella `Request`.
- Non esiste una negoziazione di capacità tra client e server.

Implicazione Modifiche ai campi del payload o ai nomi delle operazioni (`RequestType`) possono rompere la compatibilità tra versioni diverse di client e server.

6.1.2 Limitazione “una richiesta = una riga”

Il framing si basa su `readLine()`, quindi:

- una richiesta deve essere serializzata senza `newline`;
- non ci sono meccanismi nativi per payload multilinea o streaming.

Implicazione Messaggi grandi (liste molto estese di libri/recensioni) aumentano la latenza e il rischio di problemi di buffer, inoltre non esiste un chunking controllato.

6.1.3 Assenza di identificatore di correlazione (correlation id)

- Le richieste non includono un `requestId`.
- Le risposte non possono essere correlate in modo robusto se, in futuro, si introducesse concorrenza *intra-connessione* o invii asincroni.

Implicazione Oggi non è un problema perché le richieste sullo stesso socket sono gestite in modo sequenziale; tuttavia limita l'evoluzione verso modelli asincroni.

6.2 Limiti di sicurezza

6.2.1 Trasporto non cifrato (assenza di TLS)

La comunicazione avviene su TCP puro:

- nessuna cifratura del traffico;
- nessuna verifica dell'identità del server (no certificato).

Implicazione In reti non fidate, credenziali e dati possono essere intercettati (sniffing) o manipolati (MITM).

6.2.2 Gestione credenziali: password trattata come valore “hash” ma non hashata

Nel database la colonna `password_hash` suggerisce un hash, ma:

- il server salva e confronta la stringa ricevuta dal client direttamente (`UserDAO.register/login`);
- non è presente hashing (es. `bcrypt/scrypt/Argon2`) né salting.

Implicazione

- se il DB viene compromesso, le password risultano immediatamente esposte;
- l'utente può riutilizzare password su altri servizi, amplificando il rischio.

6.2.3 Assenza di gestione sessione e autorizzazione robusta

Dopo il login, il client utilizza un `userId` numerico nelle richieste successive. Non esiste:

- un token di sessione firmato;
- una scadenza della sessione;
- un meccanismo server-side di associazione socket → utente.

Implicazione Se un client malevolo conoscesse un `userId` valido, potrebbe tentare operazioni impersonando quell'utente (il server non ha un controllo crittografico sull'identità della richiesta).

6.2.4 Validazioni input parziali

Sono presenti alcune validazioni (es. campi obbligatori e lunghezza del codice fiscale), ma:

- mancano controlli di formato per email;
- mancano vincoli applicativi espliciti su lunghezza di note/descrizioni prima dell'inserimento;
- alcuni errori SQL possono propagare come messaggi generici.

Implicazione Input anomali possono produrre errori runtime o messaggi poco user-friendly; inoltre il sistema è più fragile su dati non “puliti”.

6.3 Limiti di concorrenza e scalabilità

6.3.1 Thread pool non limitato (`newCachedThreadPool`)

Il server usa un `ExecutorService` cached:

- crea nuovi thread se la domanda cresce;
- non impone un limite massimo esplicito.

Implicazione Con molte connessioni simultanee o client che mantengono socket aperti, si può arrivare a:

- consumo eccessivo di memoria (stack dei thread);
- saturazione CPU per context switch;
- degrado progressivo della reattività del server.

6.3.2 Connessione JDBC unica condivisa tra thread

`DBManager` mantiene una singola `Connection` statica usata da tutti i thread.

Implicazione

- JDBC non garantisce in generale thread-safety di una singola `Connection` condivisa;
- sotto carico, la concorrenza su risorsa condivisa può generare contese o comportamenti non deterministici.

6.3.3 Assenza di transazioni per operazioni multi-step

Alcune operazioni sono composte da più statement. In particolare, l'inserimento suggerimenti (`SuggestionDAO.inserisci`) esegue:

1. DELETE dei suggerimenti esistenti per (utente, libro)
2. INSERT batch dei nuovi suggerimenti

Implicazione In presenza di chiamate concorrenti per la stessa coppia (utente, libro):

- è possibile una condizione di **lost update** (una richiesta cancella i dati inseriti dall'altra);
- non essendoci transazione esplicita, non c'è atomicità garantita del “replace”.

6.3.4 Query senza paginazione (liste potenzialmente grandi)

Ricerca con `searchByTitle("")` restituisce **tutti** i libri.

Implicazione

- tempi di risposta e payload crescono linearmente con la dimensione del dataset;
- uso di memoria maggiore lato server (liste) e lato client (tabella JavaFX).

6.4 Limiti del modello dati

6.4.1 Mancanza di metadati temporali

Le entità `ValutazioniLibri` e `ConsigliLibri` non includono timestamp (es. `created_at`, `updated_at`):

- non è possibile ordinare recensioni per data;
- non è possibile misurare trend nel tempo (es. popolarità recente dei consigli).

6.4.2 Campo note limitato a 256 caratteri

In `ValutazioniLibri.note` è definito `VARCHAR(256)`.

Implicazione Recensioni lunghe non sono supportate; se il client invia un testo più lungo, si può incorrere in errore SQL o troncamento a seconda del comportamento del driver/DB.

6.4.3 Vincolo “max 3 suggerimenti” non è garantito a livello DB

Il limite è applicato nel codice (`if (suggeriti.size() > 3) throw ...`), ma non esiste:

- un vincolo nativo nel database;
- una procedura/trigger che lo enforcement anche per accessi esterni.

Implicazione Se altre componenti scrivessero direttamente sul DB, il vincolo potrebbe essere violato.

6.4.4 Nessuna prevenzione di suggerimenti duplicati nel payload

La PK impedisce duplicati identici (stessa tripla), ma:

- se il client invia due volte lo stesso `suggerito_libro_id` nella lista, l'inserimento batch può fallire per violazione di chiave primaria;
- non esiste un pre-filtraggio applicativo (deduplica) prima del batch.

6.5 Limiti funzionali (feature non implementate o semplificate)

L'applicazione implementa un set coerente di funzioni base (ricerca, librerie, valutazioni, suggerimenti), ma presenta alcune assenze tipiche di un prodotto completo.

6.5.1 Gestione librerie parziale

Sono disponibili:

- creazione libreria;
- aggiunta libri a libreria;
- lista librerie e libri in libreria.

Non risultano previste operazioni dedicate a:

- rimozione libro da libreria;
- eliminazione o rinomina libreria;
- gestione conflitti user-friendly sul vincolo `UNIQUE(utente_id, nome)` (nome duplicato).

6.5.2 Recensioni: una per utente e libro, senza storico

Il sistema usa upsert su (`utente_id`, `libro_id`):

- un utente può aggiornare la recensione;
- non esiste uno storico versionato delle modifiche.

6.5.3 Assenza di funzionalità di recupero account

Non sono presenti meccanismi di:

- reset password;
- conferma email;
- gestione profilo utente (modifica dati).

6.6 Limiti di usabilità, performance percepita e portabilità del client

6.6.1 Chiamate di rete sincrone su thread della UI

Nel client JavaFX, diverse operazioni invocano direttamente `ClientConnection.send(...)` all'interno di handler UI.

Implicazione Durante chiamate lente (rete o DB):

- la UI può risultare non reattiva (freeze temporaneo);
- l'utente percepisce latenza maggiore.

Una soluzione tipica sarebbe spostare le chiamate in `Task/Service` JavaFX con aggiornamento UI su `Platform.runLater`.

6.6.2 Connessione socket spesso ricreata

In varie operazioni il client istanzia `new ClientConnection()` dentro un `try-with-resources`, quindi:

- apre una nuova connessione TCP per ogni operazione;
- chiude la connessione a fine metodo.

Implicazione Maggiore overhead (handshake TCP, creazione stream) e minore efficienza sotto uso intenso.

6.6.3 Portabilità JavaFX limitata da configurazione piattaforma

Nel `pom.xml` del client è impostato:

```
<javafx.platform>win</javafx.platform>
```

Implicazione Il progetto è configurato per includere i runtime JavaFX nativi Windows; per eseguire su Linux/macOS è necessario modificare la proprietà a `linux` o `mac` e rigenerare l'artefatto.

6.7 Limiti operativi di deployment

6.7.1 Dipendenza da PostgreSQL e privilegi di creazione database

All'avvio, il server può tentare la creazione automatica del DB se mancante (con query su `pg_database` e `CREATE DATABASE`).

Implicazione

- in ambienti gestiti (DBaaS, utenze limitate) la creazione automatica può fallire per permessi insufficienti;
- è necessario predisporre manualmente il database o fornire credenziali con privilegi adeguati.

6.7.2 Seed CSV non garantisce completezza e segnala poco gli errori

Il seed:

- ignora righe malformate senza log dettagliato;
- implementa un parser CSV semplificato (gestisce quote ma non tutti i casi limite, es. escape avanzati).

Implicazione In caso di dataset non perfettamente conforme, il numero di record importati può risultare inferiore senza indicazioni precise su quali righe siano state scartate.

6.8 Sintesi dei limiti principali

I limiti più rilevanti, in ottica di evoluzione del progetto, sono:

- **sicurezza:** trasporto non cifrato e credenziali non hashate;
- **scalabilità:** thread pool non limitato, connessione DB condivisa, assenza di paginazione;
- **coerenza concorrente:** operazioni multi-step senza transazioni (suggerimenti);
- **UX:** operazioni di rete sincrone e potenziale freeze della UI;
- **portabilità:** JavaFX configurato di default per Windows.